

DATA REPRESENTATION

Data representation refers to the process of converting raw numbers and records into a format that is easier for humans to interpret. When data is displayed visually through charts, graphs, or diagrams, patterns and anomalies that are difficult to spot in raw tables immediately become apparent.

Python offers a rich ecosystem for data representation. Among the most widely used libraries are Plotly, Bokeh, Altair, Matplotlib, and Seaborn. This guide focuses on Matplotlib and Seaborn, offering beginner-friendly explanations, code walkthrough, and comparisons.

MATPLOTLIB

Matplotlib was first released in 2003 by John D. Hunter as a free, open-source plotting library. It was designed to replicate MATLAB-style graphics in Python. Today it remains one of the most foundational plotting tools in the scientific Python stack, with active maintenance under NumFOCUS.

A Matplotlib figure is composed of four main structural components:

1. Figure
2. Axes
3. Axis
4. Artists

Figure

The Figure serves as the overall canvas or window in which all visual elements are rendered. You can embed one or more Axes objects within a single Figure to create multi-panel layouts, commonly used for side-by-side comparisons.

Axes

An Axes object defines the actual drawing area — the coordinate space where data is plotted. Each Axes has its own x- and y-axis labels, ticks, title, and legend. The methods `set_xlabel()` and `set_ylabel()` are used to assign descriptive axis labels.

Axis

The Axis object is a child of Axes and manages the numerical range displayed along each axis. It controls tick placement, tick formatting, and axis limits, allowing fine-tuned control over how the scale appears.

Artists

Every visual element drawn inside the figure — lines, text, markers, filled regions, images, patches — is classified as an Artist. These are the raw building blocks that Matplotlib renders onto the canvas.

PYPLOT

Pyplot is the primary interface module within Matplotlib. It wraps much of the complexity of Matplotlib into a simple, MATLAB-like function-based interface. Pyplot supports chart types including line graphs, bar charts, scatter diagrams, pie charts, histograms, and filled area plots.

Pyplot is imported with the following standard alias:

```
1 import matplotlib.pyplot as plt
```

NumPy is typically imported alongside Matplotlib for efficient numerical data generation:

```
1 import numpy as np
```

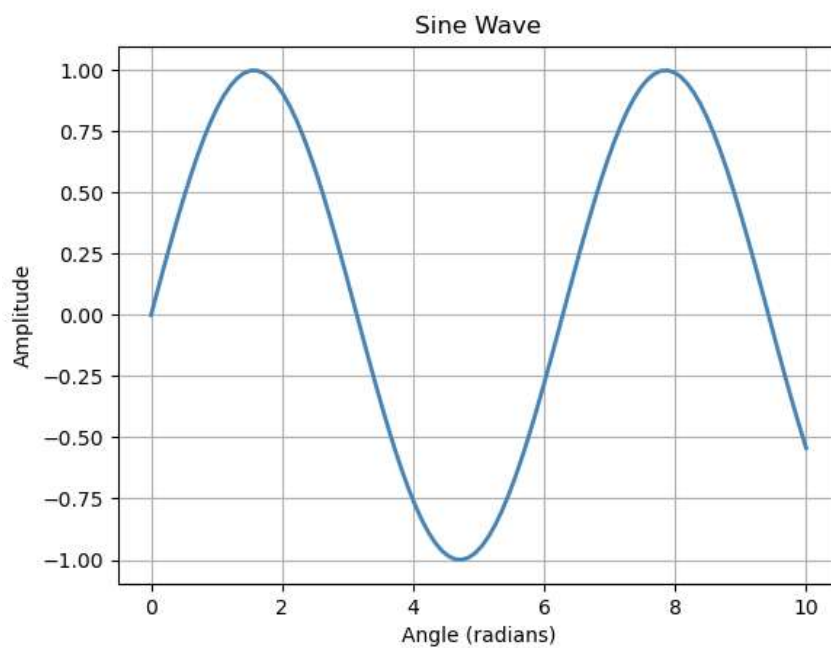
LINE CHART

A line chart connects ordered data points with straight line segments. It is the default chart type produced by Matplotlib's `plot()` function and is most commonly used to visualize trends over time or continuous numerical ranges.

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 10, 100)
5 y = np.sin(x)
6
7 plt.plot(x, y, color='steelblue', linewidth=2)
8 plt.title('Sine Wave')
9 plt.xlabel('Angle (radians)')
10 plt.ylabel('Amplitude')
11 plt.grid(True)
12 plt.show()
```

Output:



BAR GRAPH

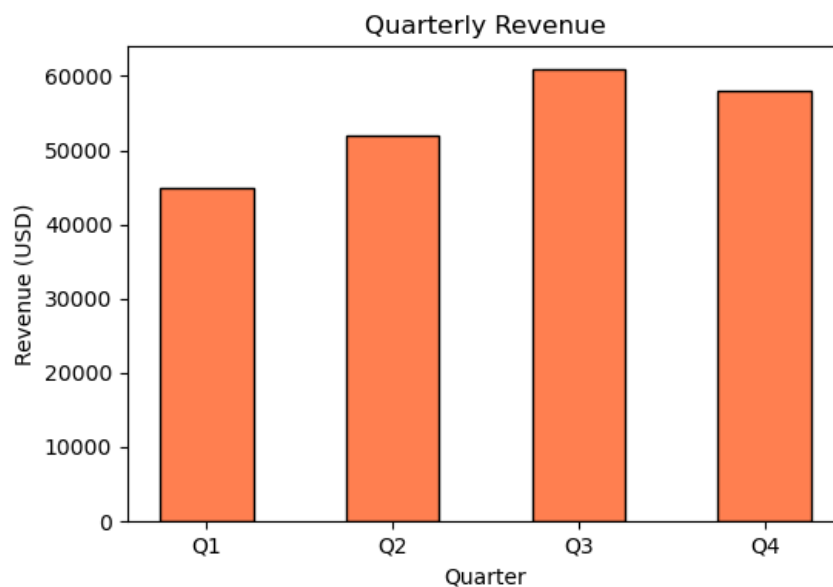
A bar graph uses rectangular bars of varying heights to compare discrete categories. Each bar's height corresponds to the magnitude of the value it represents. The `bar()` function in Matplotlib is used to produce this chart type.

Syntax: `bar(x, height, width, color, align)`

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 categories = ['Q1', 'Q2', 'Q3', 'Q4']
5 revenue = [45000, 52000, 61000, 58000]
6
7 plt.figure(figsize=(6, 4))
8 plt.bar(categories, revenue, color='coral', width=0.5, edgecolor='black')
9 plt.title('Quarterly Revenue')
10 plt.xlabel('Quarter')
11 plt.ylabel('Revenue (USD)')
12 plt.show()
```

Output:



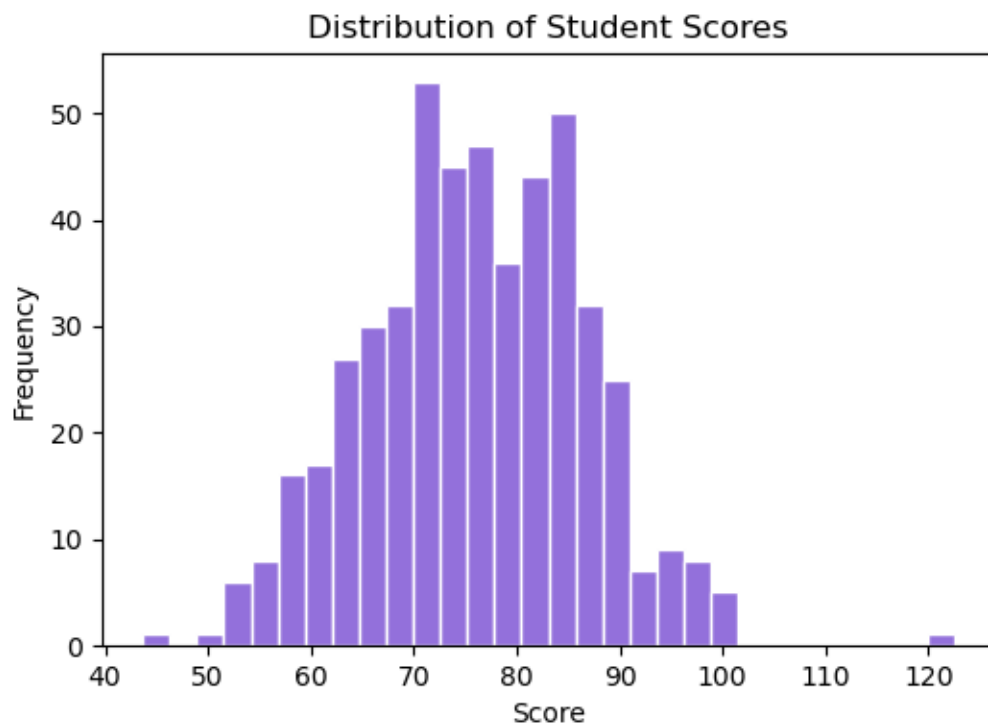
HISTOGRAM

A histogram groups continuous data into intervals called bins and displays the count (or frequency) of observations falling into each bin. Unlike bar charts, histograms represent a distribution rather than comparisons across categories. The `hist()` function is used for this purpose.

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 data = np.random.normal(loc=75, scale=10, size=500)
5 plt.figure(figsize=(6, 4))
6 plt.hist(data, bins=30, color='mediumpurple', edgecolor='white')
7 plt.title('Distribution of Student Scores')
8 plt.xlabel('Score')
9 plt.ylabel('Frequency')
10 plt.show()
```

Output:



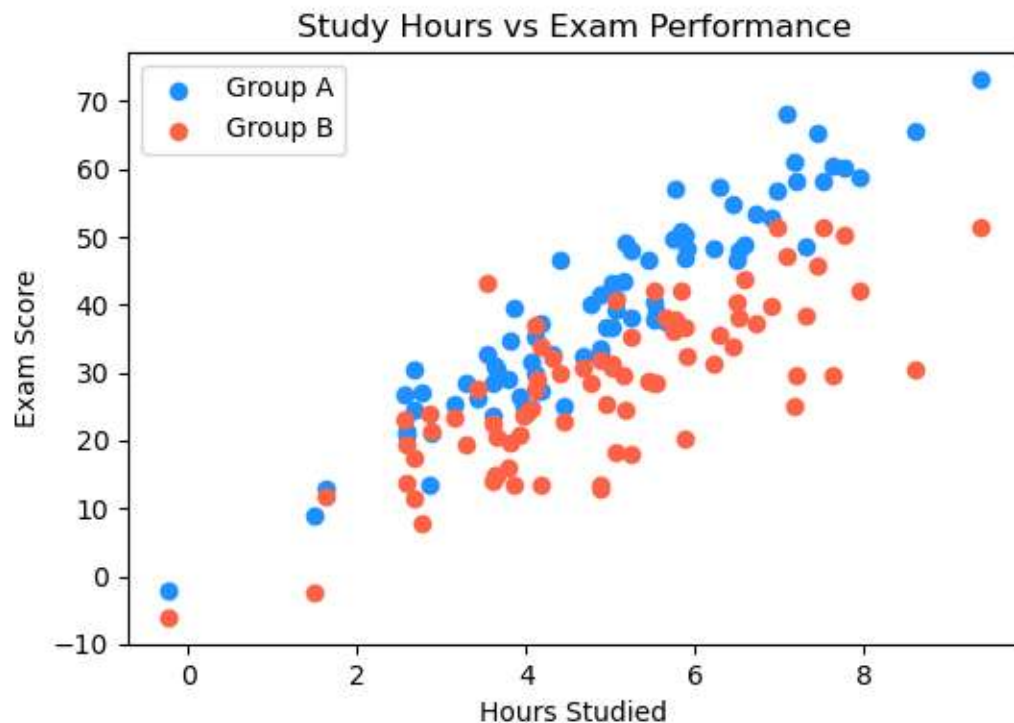
SCATTER PLOT

Scatter plots use individual dots positioned at (x, y) coordinates to reveal relationships or clusters between two numerical variables. Multiple datasets can be overlaid in the same plot using different marker colours, distinguished via a legend.

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 hours_studied = np.random.normal(5, 1.5, 80)
5 exam_score_a = hours_studied * 8 + np.random.normal(0, 5, 80)
6 exam_score_b = hours_studied * 6 + np.random.normal(0, 8, 80)
7
8 plt.figure(figsize=(6, 4))
9 plt.scatter(hours_studied, exam_score_a, c='dodgerblue', label='Group A')
10 plt.scatter(hours_studied, exam_score_b, c='tomato', label='Group B')
11 plt.legend()
12 plt.title('Study Hours vs Exam Performance')
13 plt.xlabel('Hours Studied')
14 plt.ylabel('Exam Score')
15 plt.show()
```

Output:



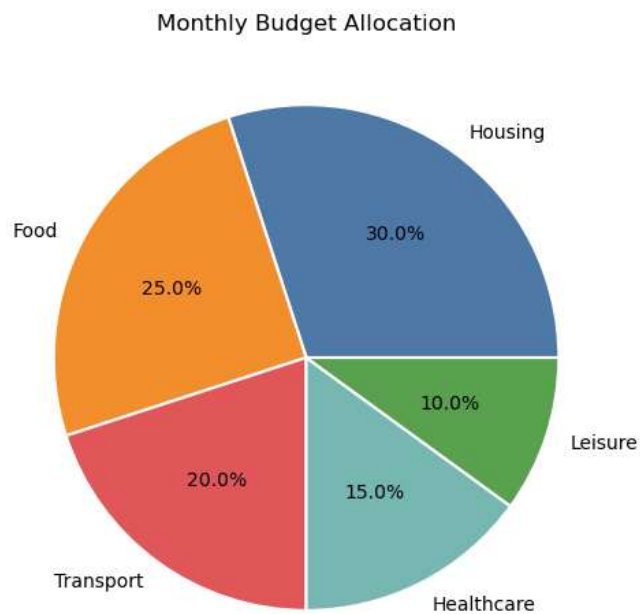
PIE CHART

A pie chart divides a circle into sectors, with each sector's arc length proportional to its share of the total. It is best suited for showing part-to-whole relationships where there are only a few distinct categories.

Code snippet:

```
1 from matplotlib import pyplot as plt
2
3 budget = [30, 25, 20, 15, 10]
4 labels = ['Housing', 'Food', 'Transport', 'Healthcare', 'Leisure']
5 colors = ['#4E79A7', '#F28E2B', '#E15759', '#76B7B2', '#59A14F']
6
7 plt.figure(figsize=(6, 6))
8 plt.pie(budget, labels=labels, colors=colors,
9         autopct='%1.1f%%',
10        wedgeprops={'edgecolor': 'white', 'linewidth': 1.5})
11 plt.title('Monthly Budget Allocation')
12 plt.show()
```

Output:



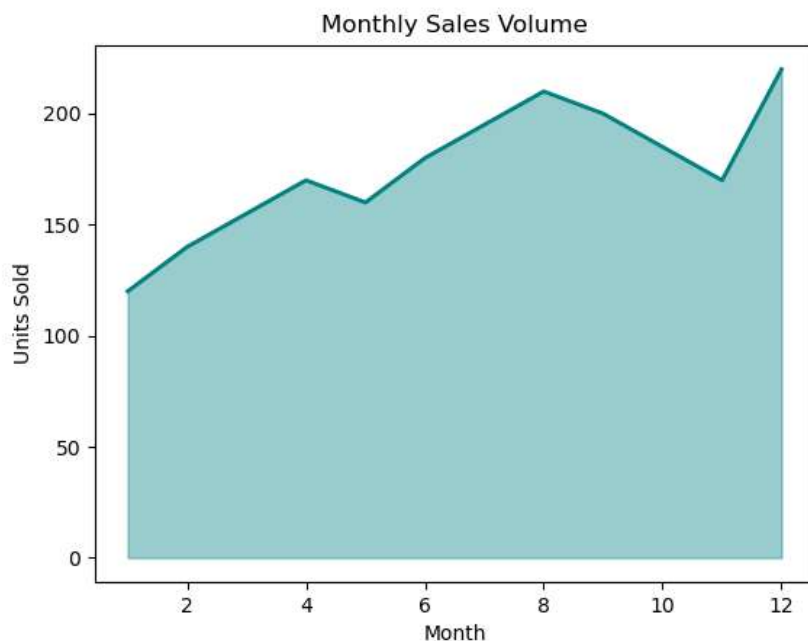
AREA CHART

An area chart fills the region between a plotted line and the x-axis, emphasising volume rather than just trend direction. Two approaches are available: `fill_between()` for a single series, and `stackplot()` for multiple stacked series.

Using `fill_between()`:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 months = np.arange(1, 13)
5 sales = [120,140,155,170,160,180,195,210,200,185,170,220]
6
7 plt.fill_between(months, sales, alpha=0.4, color='teal')
8 plt.plot(months, sales, color='teal', linewidth=2)
9 plt.title('Monthly Sales Volume')
10 plt.xlabel('Month')
11 plt.ylabel('Units Sold')
12 plt.show()
```

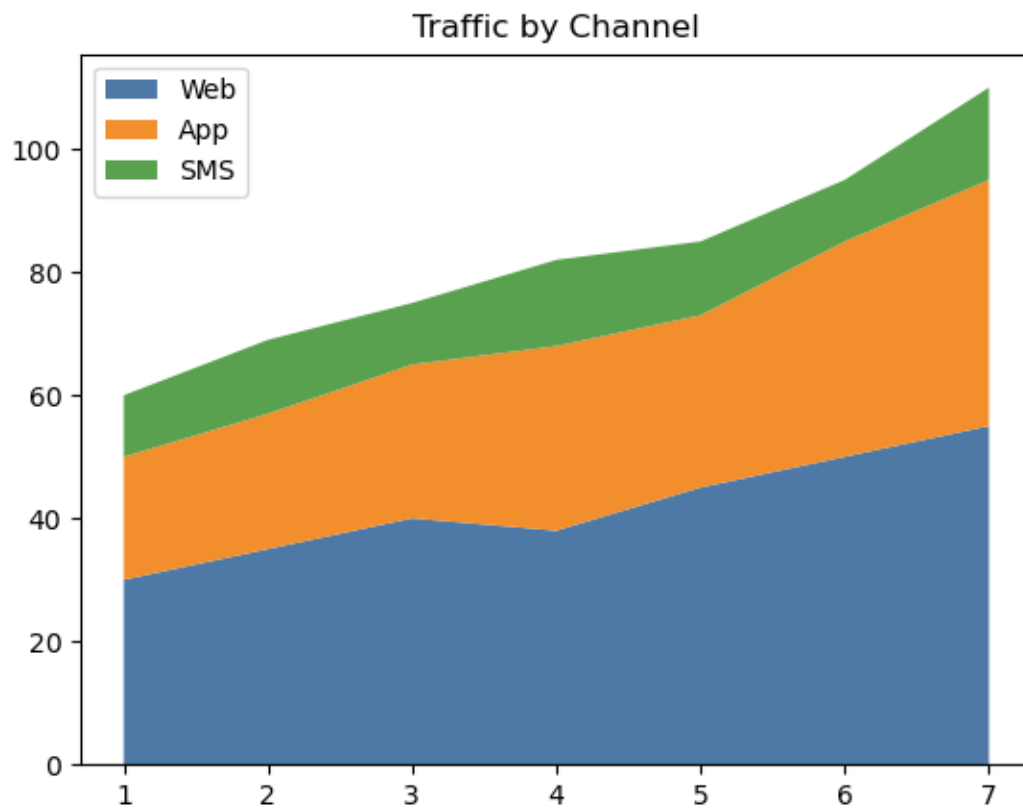
Output:



Using stackplot():

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.arange(1, 8)
5 web = [30, 35, 40, 38, 45, 50, 55]
6 app = [20, 22, 25, 30, 28, 35, 40]
7 sms = [10, 12, 10, 14, 12, 10, 15]
8
9 plt.stackplot(x, web, app, sms,
10             labels=['Web', 'App', 'SMS'],
11             colors=['#4E79A7', '#F28E2B', '#59A14F'])
12 plt.legend(loc='upper left')
13 plt.title('Traffic by Channel')
14 plt.show()
```

Output:



SEABORN

Seaborn is a statistical data visualisation library built on top of Matplotlib. While Matplotlib provides low-level primitives, Seaborn adds high-level abstractions designed specifically for exploring statistical relationships in data. It integrates seamlessly with Pandas DataFrames and applies polished default themes automatically.

Seaborn organises its plots into five functional categories:

1 Relational Plots:

- `scatterplot()`
- `lineplot()`
- `relplot()`

2 Categorical Plots:

- `barplot()`
- `countplot()`
- `boxplot()`
- `violinplot()`
- `stripplot()`
- `catplot()`

3 Distribution Plots:

- `histplot()`
- `kdeplot()`
- `ecdfplot()`
- `rugplot()`

4 Regression Plots:

- `regplot()`
- `lmpplot()`
- `residplot()`

5 Matrix Plots:

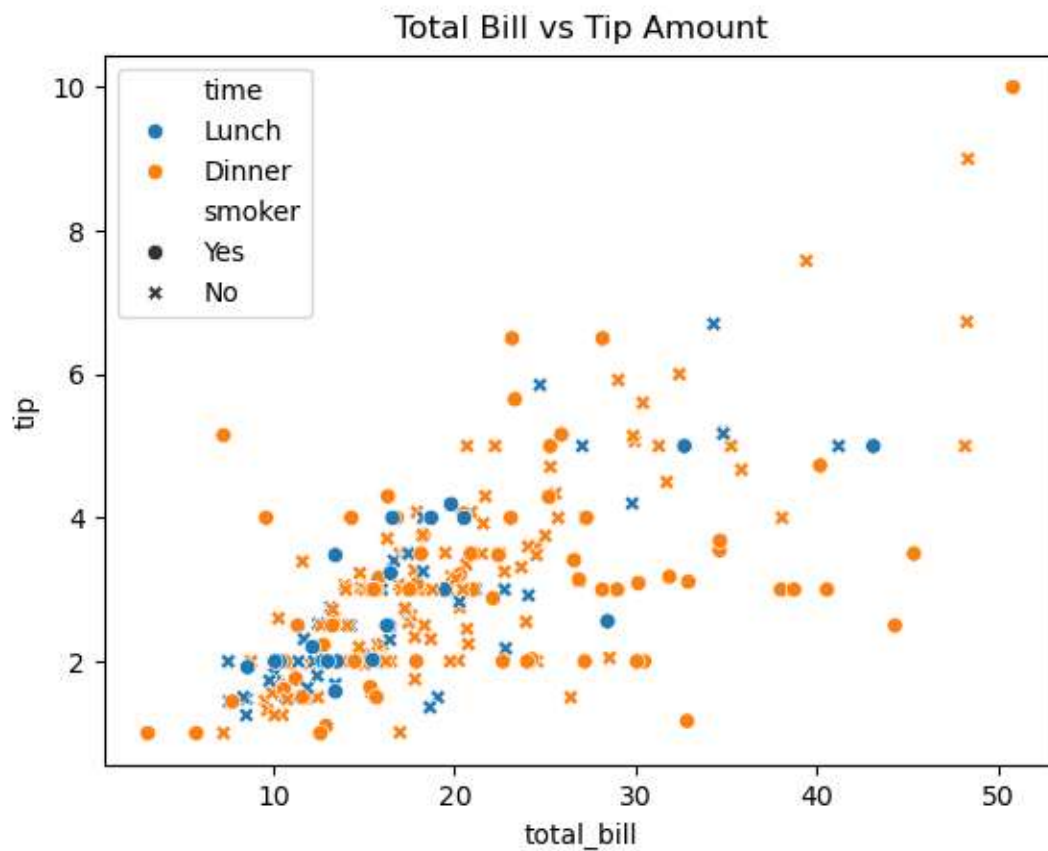
- `heatmap()`
 - `clustermap()`
-

Sample code for selected Seaborn charts is provided below. All examples use the built-in 'tips' dataset (restaurant tip records).

SCATTER PLOT

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.scatterplot(x='total_bill', y='tip', hue='time', style='smoker', data=df)
6 plt.title('Total Bill vs Tip Amount')
7 plt.show()
```

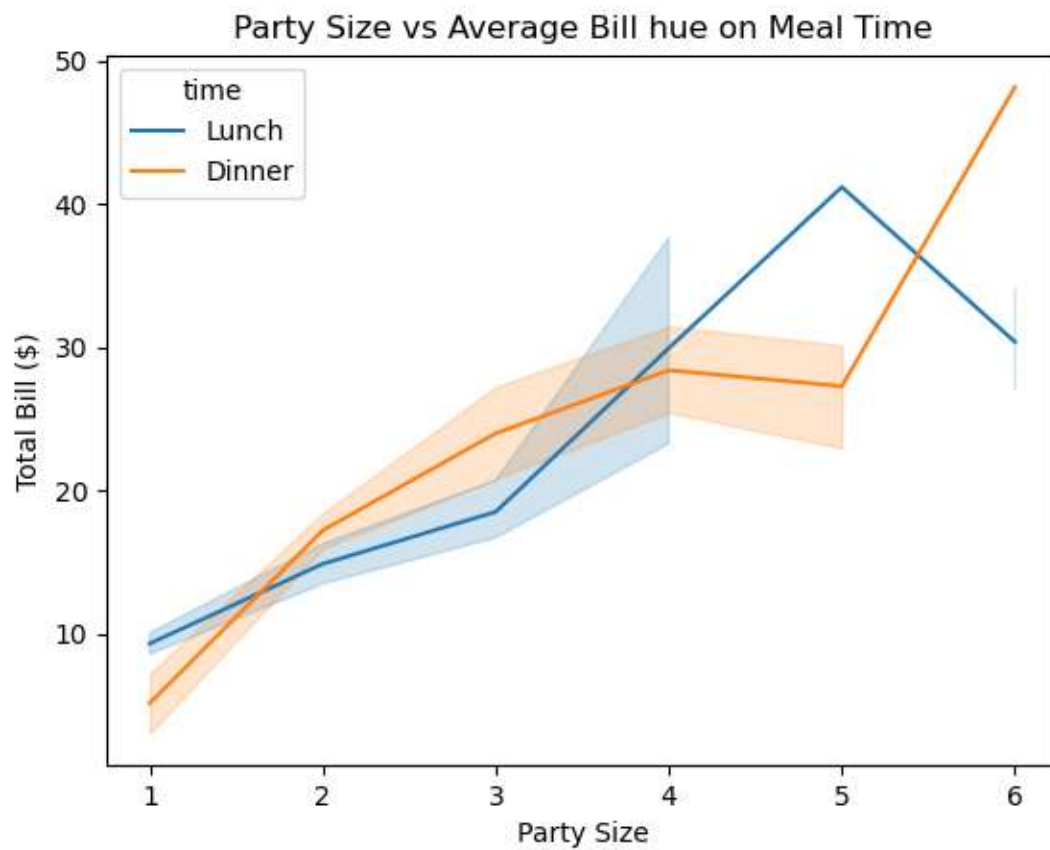
Output:



LINE PLOT

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 df = sns.load_dataset('tips')
6 sns.lineplot(x='size', y='total_bill', hue='time', data=df)
7 plt.title('Party Size vs Average Bill hue on Meal Time')
8 plt.xlabel('Party Size')
9 plt.ylabel('Total Bill ($)')
10 plt.show()
```

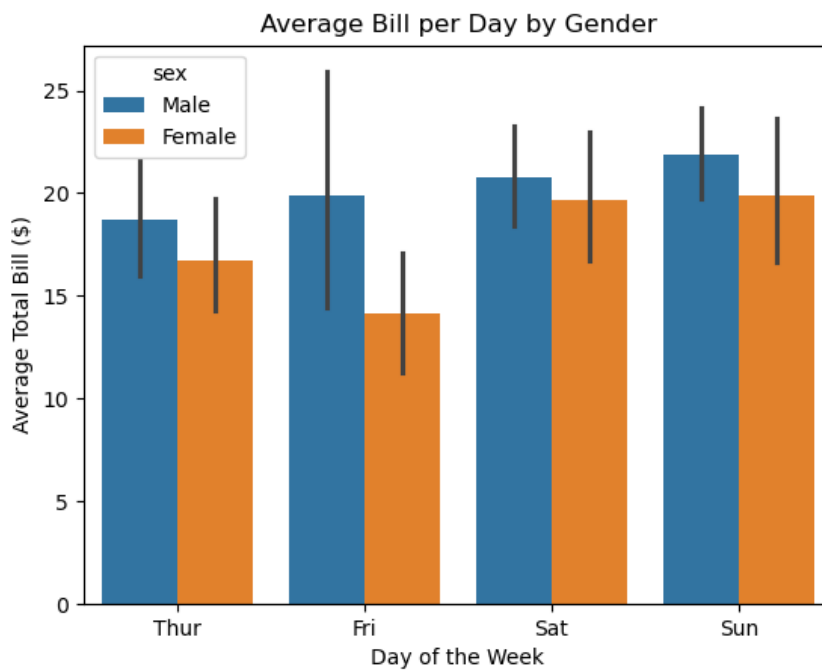
Output:



BAR PLOT

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.barplot(x='day', y='total_bill', hue='sex', data=df)
6 plt.title('Average Bill per Day by Gender')
7 plt.xlabel('Day of the Week')
8 plt.ylabel('Average Total Bill ($)')
9 plt.show()
```

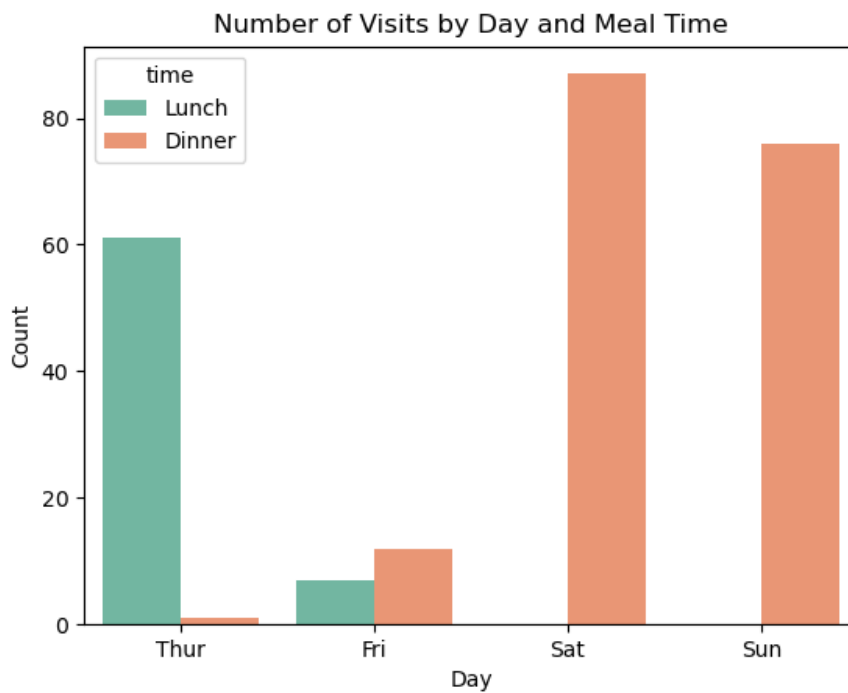
Output:



COUNT PLOT

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.countplot(x='day', hue='time', data=df, palette='Set2')
6 plt.title('Number of Visits by Day and Meal Time')
7 plt.xlabel('Day')
8 plt.ylabel('Count')
9 plt.show()
```

Output:

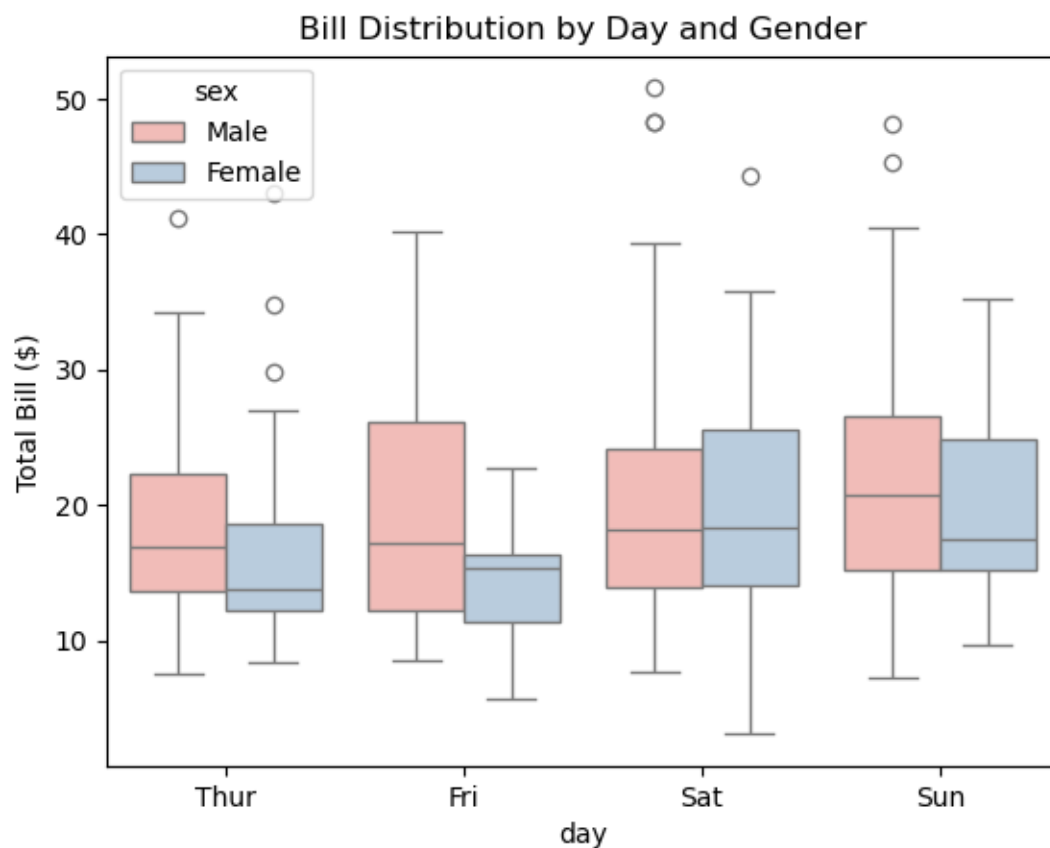


BOX PLOT

A box plot (box-and-whisker plot) compactly summarises a numeric distribution. The central box spans the interquartile range (IQR: Q1 to Q3), the line inside marks the median, and whiskers extend to 1.5 times the IQR. Points beyond the whiskers are plotted individually as outliers.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.boxplot(x='day', y='total_bill', hue='sex', data=df, palette='Pastell1')
6 plt.title('Bill Distribution by Day and Gender')
7 plt.xlabel('Day')
8 plt.ylabel('Total Bill ($)')
9 plt.show()
```

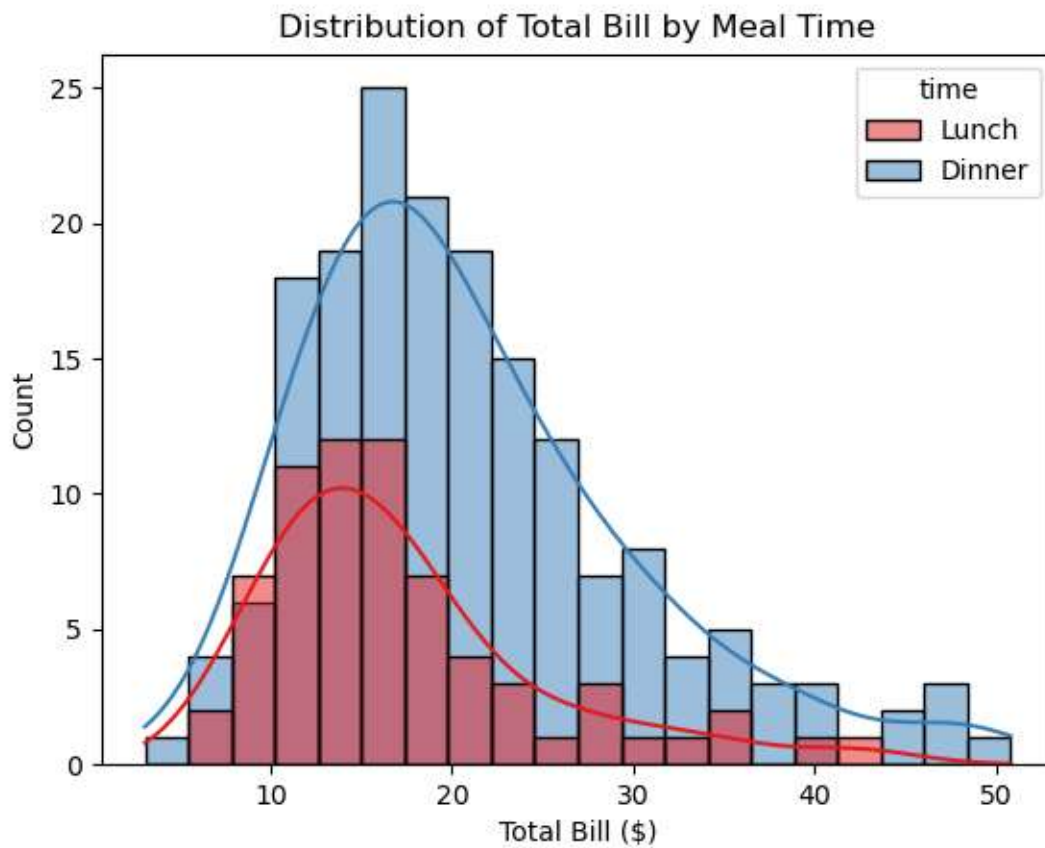
Output:



HISTOGRAM

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.histplot(x='total_bill', hue='time', data=df,
6             bins=20, kde=True, palette='Set1')
7 plt.title('Distribution of Total Bill by Meal Time')
8 plt.xlabel('Total Bill ($)')
9 plt.ylabel('Count')
10 plt.show()
```

Output:

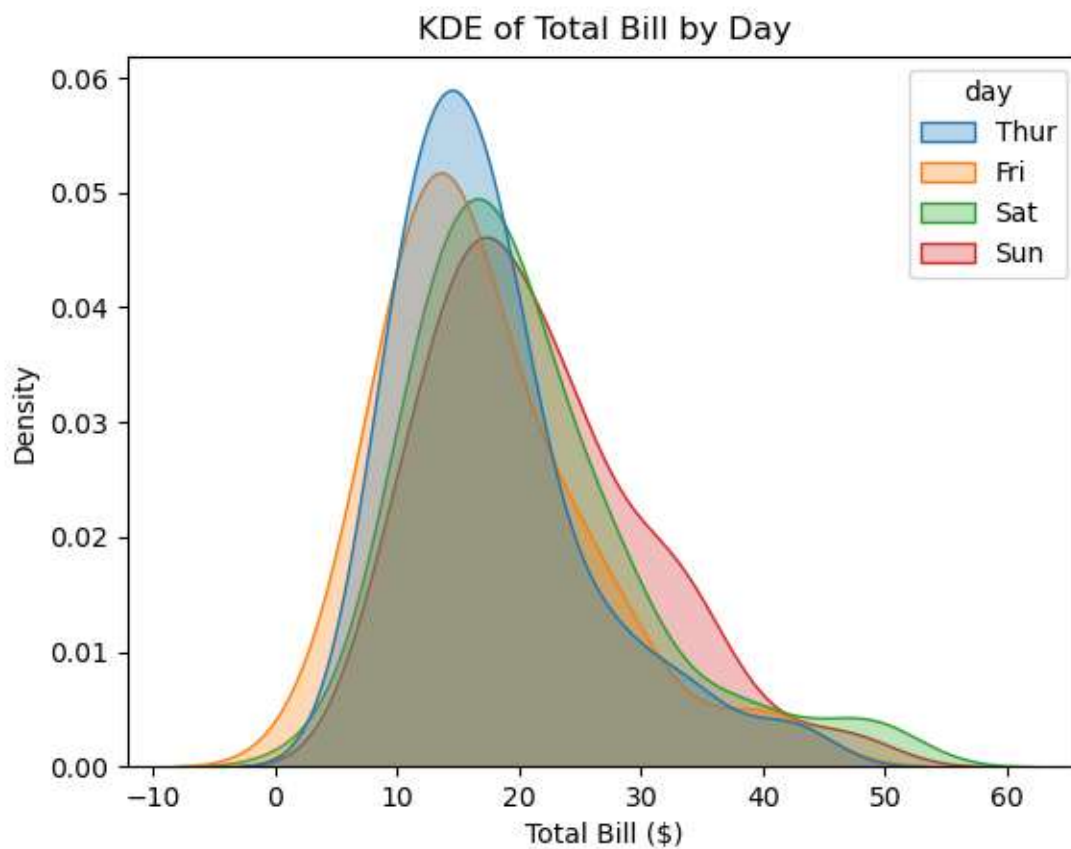


KDEPLOT

KDE stands for Kernel Density Estimate. A KDE plot fits a smooth continuous curve to a distribution of data, effectively creating a probability density function from sample observations. It is particularly useful when comparing distributions across multiple groups on a single chart.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df = sns.load_dataset('tips')
5 sns.kdeplot(x='total_bill', hue='day', data=df,
6             fill=True, alpha=0.3, common_norm=False)
7 plt.title('KDE of Total Bill by Day')
8 plt.xlabel('Total Bill ($)')
9 plt.show()
```

Output:



HEATMAP

A heatmap encodes numeric values as colour intensities in a matrix grid. It is especially powerful for visualising correlation matrices, showing how pairs of numerical features co-vary. Warm colours typically indicate high positive values and cool colours indicate low or negative values.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 df = sns.load_dataset('tips')
6 numeric_df = df.select_dtypes(include='number')
7 corr = numeric_df.corr()
8
9 plt.figure(figsize=(6, 5))
10 sns.heatmap(corr, annot=True, fmt='.2f',
11            cmap='RdYlGn', linewidths=0.5)
12 plt.title('Correlation Heatmap — Tips Dataset')
13 plt.show()
```

Output:

